

Using **DBML (Database Markup Language)** on dbdiagram.io is an industry-standard best practice. Designing schemas visually before writing Mongoose code prevents circular references, missing indexes, and unoptimized relational queries.

Below is the production-ready DBML diagram code formatted with strict **camelCase** naming conventions, followed by the exact file-naming blueprint for your backend.

1. DBML Diagram Code for dbdiagram.io

Copy the code below, open dbdiagram.io, and paste it into the editor to generate your interactive Entity Relationship Diagram instantly:

Code snippet

```
// =====  
// 2D AI Interactive Simulation Canvas Schema  
// =====
```

```
Table users {  
  id string [pk, note: 'MongoDB ObjectId']  
  userName string [not null]  
  email string [not null, unique]  
  passwordHash string [not null]  
  avatarUrl string  
  refreshToken string  
  createdAt datetime  
  updatedAt datetime  
}
```

```
Table workspaces {  
  id string [pk, note: 'MongoDB ObjectId']  
  userId string [not null, ref: > users.id]  
  title string [not null]  
  slug string [not null, unique]  
  description string  
  diagramType string [not null, note: 'flowchart | simulationTree | trackingGauge | codeLogic']  
  isPublished boolean [default: false]  
  createdAt datetime  
  updatedAt datetime  
}
```

```
Table canvasGraphs {  
  id string [pk, note: 'MongoDB ObjectId']  
  workspaceId string [not null, unique, ref: - workspaces.id]  
  nodesData json [not null, note: 'Array of nodes: {id, type, label, position, formula, style}']  
}
```

```

edgesData json [not null, note: 'Array of edges: {id, source, target, animated, condition}']
globalMetrics json [note: 'Cumulative simulation tracking counters and state variables']
viewport json [note: '{x: number, y: number, zoom: number}']
updatedAt datetime
}

```

```

Table graphVersions {
  id string [pk, note: 'MongoDB ObjectId']
  workspaceId string [not null, ref: > workspaces.id]
  versionNumber int [not null]
  changeSummary string [not null]
  stateSnapshot json [not null, note: 'Full serialized graph tree at this iteration']
  diffTree json [note: 'Target parameter mutations only']
  createdBy string [not null, ref: > users.id]
  createdAt datetime
}

```

```

Table aiOrchestrations {
  id string [pk, note: 'MongoDB ObjectId']
  workspaceId string [not null, ref: > workspaces.id]
  bullmqJobId string [note: 'Redis queue task identifier']
  inputType string [not null, note: 'structuredForm | rawCode']
  rawInput text [not null, note: 'Form payload or raw code snippet']
  promptPayload text [not null]
  responsePayload json [note: 'Zod-validated JSON graph schema']
  status string [not null, default: 'queued', note: 'queued | processing | completed | failed']
  errorMessage text
  createdAt datetime
}

```

- **users:** Manages authentication, hashed credentials, and refresh tokens.
- **workspaces:** Represents a canvas project (1 User can own many Workspaces).
- **canvasGraphs:** Stores the live, active state (nodes, coordinates, formulas, edges) of a single workspace in a 1:1 relationship.
- **graphVersions:** Stores undo/redo history snapshots and non-destructive diffs.
- **aiOrchestrations:** Tracks BullMQ background jobs, prompts, raw code inputs, and validated AI outputs.

2. Backend File Structure Blueprint

To match your established MVC architecture, here is the exact mapping of models, controllers, and routers:

Entity	Model File (models/)	Controller File (controllers/)	Router File (routers/)
User	User.model.js	user.controller.js	user.router.js
Workspace	Workspace.model.js	workspace.controller.js	workspace.router.js
Canvas Graph	CanvasGraph.model.js	canvasGraph.controller.js	canvasGraph.router.js
Graph Version	GraphVersion.model.js	graphVersion.controller.js	graphVersion.router.js
AI Orchestration	AiOrchestration.model.js	aiOrchestration.controller.js	aiOrchestration.router.js

Would you like to write the Mongoose schema code for **CanvasGraph.model.js** and **Workspace.model.js** first, or build the **Zod schema** that validates the node formulas and coordinates?